

# ResolveIQ Platform Redesign

Revision 2 adds a structured knowledge layer — Product, Technology, and Version Intelligence — so recommendations reason from what the platform actually knows about Command Center/Emerge, not just what the text looks like.

**Status** Draft for final review   **Supersedes** RFC rev 1   **Blocks** Sprint 2 — held pending approval

**Changed since rev 1** §03, §05, §07, §09–§12

## oo What real data broke

Unchanged from rev 1 — restated for context.

### INGESTION

#### Binary files parsed as UTF-8

.docx, .xlsx, and .jpg uploads were decoded as plain text, polluting entity extraction and the semantic index.

### RANKING

#### Keyword overlap beat domain understanding

A Get Load Profile / firmware-DCW mismatch was out-ranked by an unrelated RF mesh timeout case on shared generic words.

### WORKFLOW

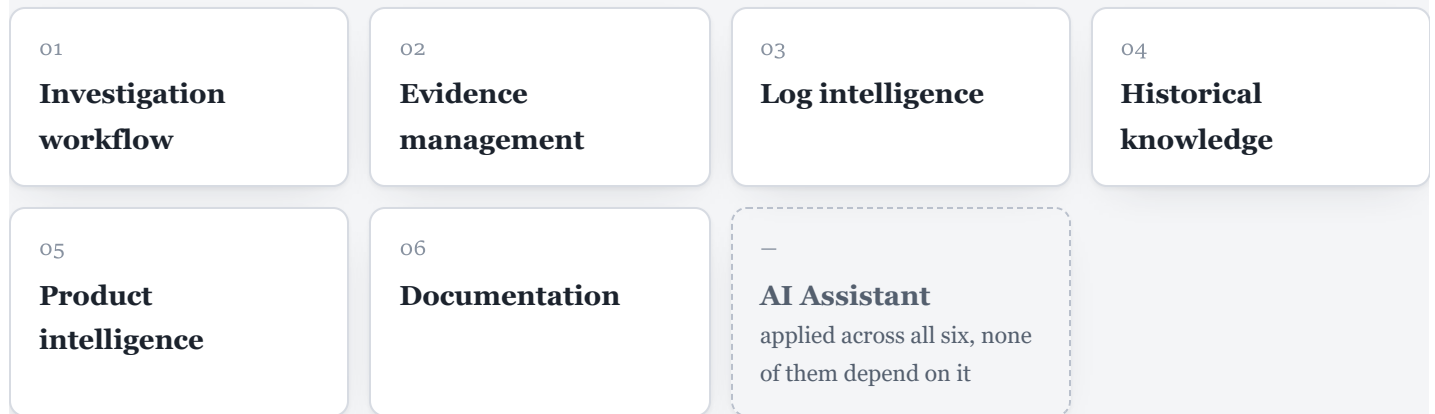
#### One scrolling page, no investigation shape

No navigation, tabs, or progress state — a single page holds a form submission, not an investigation.

## 01 Platform philosophy

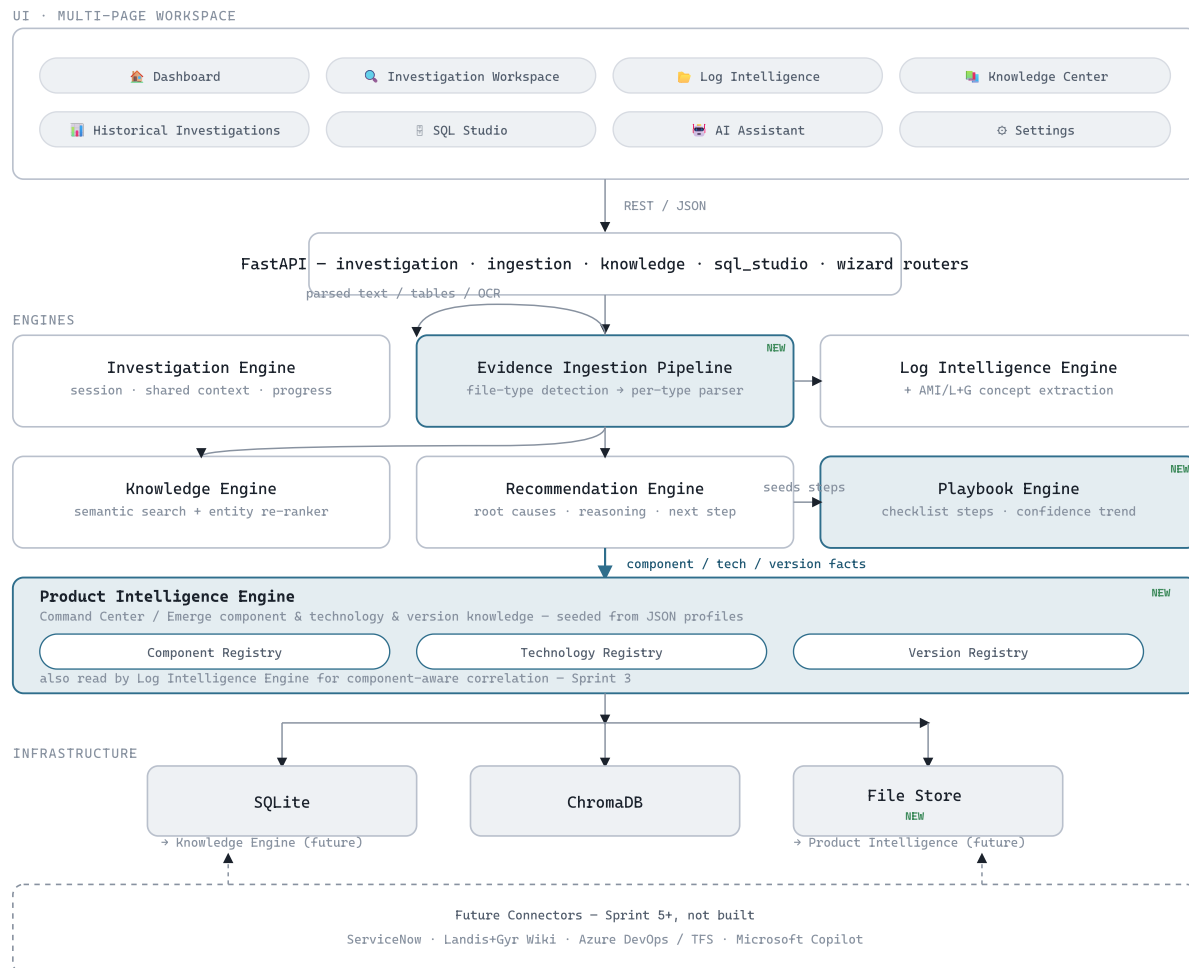
Stated explicitly because it's architectural, not aspirational: **ResolveIQ is not an AI chatbot.** It is an Investigation Intelligence Platform. AI is one capability applied across the platform — it is not the platform.

Concretely, that means the AI Assistant is one module among eight (S07, Fig. 7d), not the entry point, not the home page, and not a dependency for any other module to function. An engineer can run a full investigation — collect evidence, correlate logs, match history, consult product knowledge, resolve the case — without opening it once. The six pillars below are the product; AI drafts against them, it doesn't replace them.



## 02 Updated architecture

Rev 2 adds **Product Intelligence Engine** as a cross-cutting knowledge layer beneath the six processing engines — it doesn't process evidence, it answers structured questions the Recommendation Engine asks of it.



**Fig. 1** — Product Intelligence Engine (accent) sits beneath the processing engines and feeds the Recommendation Engine directly. It does not sit in the evidence path.

03

# Product Intelligence Engine

Your items 1, 2, and 8 — component knowledge, version intelligence, and technology profiles — are one engine here, not three. All three are the same shape (an entity → structured facts → links into logs, bugs, docs, playbooks, SQL, historical investigations) and the same consumer (the Recommendation Engine's reasoning layer). Three separate engines would triplicate that lookup/link machinery for no benefit; one engine with three registries keeps it DRY without losing any of the three concepts.

## Component Registry – one profile per architecture component

Every component named in your list — CommandProcessorHost, CommandPayloadProcessorHost, MsgProcCmdRspHost, Scheduler, Kafka, RabbitMQ, IIS, NMS, Device Hub, Collectors, Kubernetes, and more — gets a structured profile with exactly the eight fields you specified:

| CommandProcessorHost <span>Command Center · service</span>                                                                                       |                                                                                                    |
|--------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <div>RESPONSIBILITIES</div> <div>Receives meter commands from the scheduler, validates against current DCW, dispatches to the mesh network</div> | <div>RELATED COMPONENTS</div> <div>CommandPayloadProcessorHost, MsgProcCmdRspHost, Scheduler</div> |
| <div>TYPICAL FAILURES</div> <div>Firmware/DCW mismatch after upgrade without GEI; command payload built against stale DCW</div>                  | <div>RELATED LOGS</div> <div>CommandProcessorHost.log, CommandPayloadProcessorHost.log</div>       |
| <div>KNOWN BUGS</div> <div>→ known_bugs filtered by component</div>                                                                              | <div>DOCUMENTATION</div> <div>→ documentation filtered by component</div>                          |
| <div>INVESTIGATION PLAYBOOKS</div> <div>GLP/DCW mismatch triage, command timeout triage</div>                                                    | <div>COMMON SQL</div> <div>SELECT * FROM CommandLog WHERE CommandLogId = ?</div>                   |

Historical investigations aren't a ninth field — they're the Knowledge Engine's own collection, tagged by component, which the reasoning layer joins against (§05).

## Technology Registry – communication technology profiles

This is what your own real investigation text opened with — "1. Technology: (TS1, TS2, RF Mesh, RF Mesh IP, PLX, PLC, Blended, Cellular, Wi-Sun)" — so it becomes the Wizard's second question (§07, Fig. 4), not just a knowledge-base tag.

| TECHNOLOGY | COMMON ISSUES                                     | REQUIRED LOGS                                    |
|------------|---------------------------------------------------|--------------------------------------------------|
| RF Mesh    | Mesh hop degradation, router firmware regressions | Collector command log, mesh health log           |
| RF Mesh IP | IP routing table desync after topology change     | Collector command log, routing table dump        |
| Wi-SUN     | Border router handoff failures                    | Wi-SUN border router log                         |
| Cellular   | Carrier connectivity drops, SIM provisioning      | Device Hub connection log                        |
| PLC        | Line noise, phase coupling loss                   | Collector PLC signal log                         |
| Blended    | Fallback-path misrouting between technologies     | Collector command log (all technologies present) |

### Version Registry – product version awareness

Versions (Command Center 8.x, 9.x, Emerge 4.2, ...) carry their own known bugs, release notes, breaking changes, and behavior differences – the same shape as a component profile, keyed by product+version instead of component name.

| PRODUCT · VERSION  | KNOWN BEHAVIOR                                        | RELEVANT TO THIS RFC                                                                      |
|--------------------|-------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Command Center 8.x | GEI required after every firmware push; not automatic | Root cause of the GLP/DCW mismatch class of issue                                         |
| Command Center 9.x | GEI auto-triggered on firmware upgrade completion     | Same symptom in 8.x is not reproducible in 9.x – a version mismatch invalidates the match |

This is exactly what feeds the "X Different product version" line in §05 – Version Registry is what makes that a real check instead of a plausible-sounding label.

# 04 Evidence Ingestion Pipeline

Unchanged from rev 1. Direct fix for Problem 1 — nothing reaches entity extraction or the semantic index until it has been through the parser that understands its format.

| FILE TYPE                                                                                     | PARSER                   | LIBRARY                                                                        | OUTPUT                                                        |
|-----------------------------------------------------------------------------------------------|--------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------|
| <code>.log</code> <code>.txt</code> <code>.csv</code> <code>.json</code><br><code>.xml</code> | Text / structured parser | <code>stdlib</code><br><code>csv</code> / <code>json</code> / <code>xml</code> | Plain text (+ table for csv, tree for json/xml)               |
| <code>.docx</code>                                                                            | Word document parser     | <code>python-docx</code>                                                       | Paragraph & table text                                        |
| <code>.xlsx</code>                                                                            | Spreadsheet parser       | <code>openpyxl</code>                                                          | Per-sheet cells as text + structured rows                     |
| <code>.pdf</code>                                                                             | PDF parser               | <code>PyMuPDF</code> ( <code>fitz</code> )                                     | Page text, OCR fallback for scanned pages                     |
| <code>.jpg</code> <code>.png</code>                                                           | Image OCR parser         | <code>pytesseract</code> + Pillow                                              | OCR'd text, flagged low-confidence below threshold            |
| <code>.evtx</code>                                                                            | Windows Event Log parser | <code>python-evtx</code>                                                       | One structured event per record                               |
| <code>.zip</code>                                                                             | Archive parser           | <code>stdlib</code> <code>zipfile</code>                                       | Recurses per entry – each becomes its own child Evidence item |
| anything else                                                                                 | Unsupported file handler | –                                                                              | Stored as binary blob only, never decoded as UTF-8            |

**Still flagging:** OCR needs the `tesseract` binary on the host, not just a Python package — scoped as best-effort with a clear "OCR unavailable" fallback, not a hard dependency, for Sprint 2.

## 05 Recommendation quality + transparency

Two causes from rev 1 (no matching record, no specificity weighting) plus your new requirement: the engine must show its work, not just a number.

### Hybrid ranking – semantic + entity overlap

#### TODAY – SEMANTIC ONLY

- 1 **Meter comm. timeout, collector COLL-14**  
score 0.79  
shares "endpoint," "command log" – generic AMI vocabulary
- 2 **IIS 502.3 Bad Gateway**  
score 0.65

#### AFTER – HYBRID RE-RANK

- 1 **GLP / firmware-DCW mismatch case**  
**requires new record**  
score 0.91  
exact match on `command_type`, `host_component`
- 2 **Meter comm. timeout, collector COLL-14**  
score 0.58  
demoted – generic overlap only

### Recommendation transparency – your new requirement

The Recommendation Engine now produces a `MatchReasoning` list alongside every candidate, not just a confidence float. Each factor is a direct comparison against Product Intelligence Engine data (§03) – a structured diff, not a generated explanation:

- ✓ **Same component** – `CommandProcessorHost` in both this case and the matched investigation
- ✓ **Same firmware** – 13.63 in both
- ✓ **Similar historical investigation** – 91% semantic + entity match
- ✗ **Different product version** – this case is CC 9.2, matched case is CC 8.4 (§03, Version Registry)

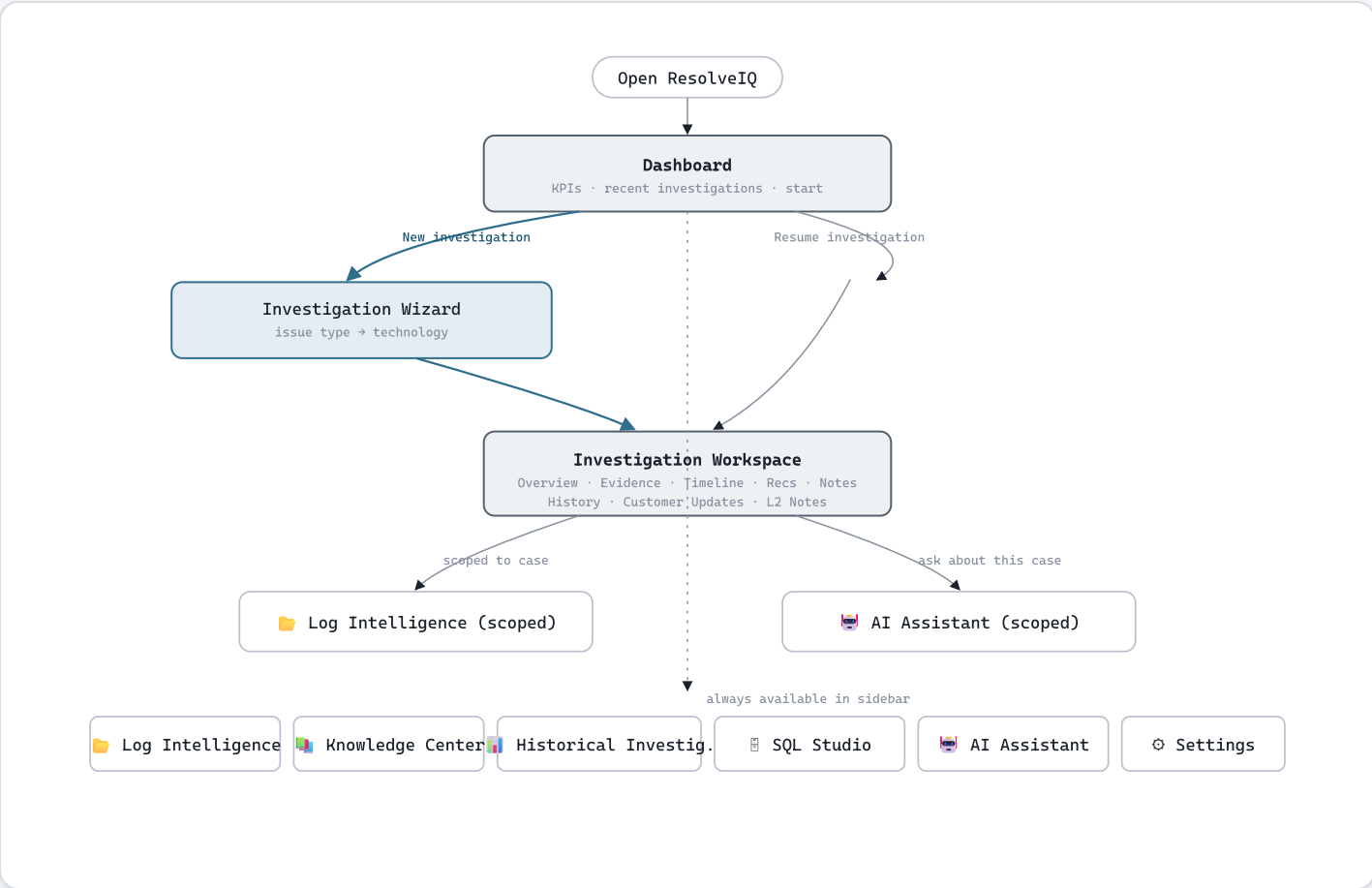
A version mismatch doesn't disqualify a match – CC 9.2's auto-GEI behavior means the *same symptom* in 8.4 may not have the *same root cause* here, so this factor should visibly lower confidence rather than being silently absorbed into one score.

### Domain concepts the Log Intelligence Engine must recognize

| CONCEPT                 | ENTITY TYPE                                              | EXAMPLE VALUE                                                        |
|-------------------------|----------------------------------------------------------|----------------------------------------------------------------------|
| Meter command           | <code>command_type</code>                                | Get Load Profile, Ping                                               |
| Processing host         | <code>host_component</code>                              | CommandProcessorHost, CommandPayloadProcessorHost, MsgProcCmdRspHost |
| Firmware / DCW pairing  | <code>firmware_version</code> / <code>dcw_version</code> | 13.66 / 13.63                                                        |
| Post-upgrade sync event | <code>gei_event</code>                                   | GEI sent / GEI missing                                               |
| Load profile config     | <code>lp_set_number</code>                               | LP Set 3                                                             |
| Product version         | <code>product_version</code>                             | CC 9.2, Emerge 4.2                                                   |

## o6 Navigation flow

Unchanged from rev 1 — the Wizard gains an internal technology step (§07, Fig. 4), but the flow between modules is the same shape.



**Fig. 2** — Accent path is the primary flow for engineers who start with only a symptom, not an ID.

## 07 UI wireframes

Two changes from rev 1: the Wizard now asks technology as well as issue type, and the Recommendations tab shows match reasoning, not just a score.

INVESTIGATION WORKSPACE — RECOMMENDATIONS TAB

Overview

Evidence

Timeline

Recommendations

Notes

History

Customer Updates

L2 Notes

Top match — 91%

✓ Same component (CommandProcessorHost)

Alt match — 58%

✓ Same firmware (13.63)

Alt match — 44%

✓ Similar historical investigation (91%)

X Different product version (CC 9.2 vs 8.4)

**Playbook**

- ☒ Verify firmware/DCW
- ☒ Check GEI sent
- ☐ Compare GLP payload

**Fig. 3** — Every match ships its reasoning inline, not behind a tooltip.

INVESTIGATION WIZARD — STEP 1 OF 2

"What are you investigating?"

UI Issue

Low Reads

Command Failure

Meter Communication

Database

Kafka

RabbitMQ

Kubernetes

Network

Unknown

INVESTIGATION WIZARD — STEP 2 OF 2

NEW

"Which technology?"

RF Mesh

RF Mesh IP

Wi-SUN

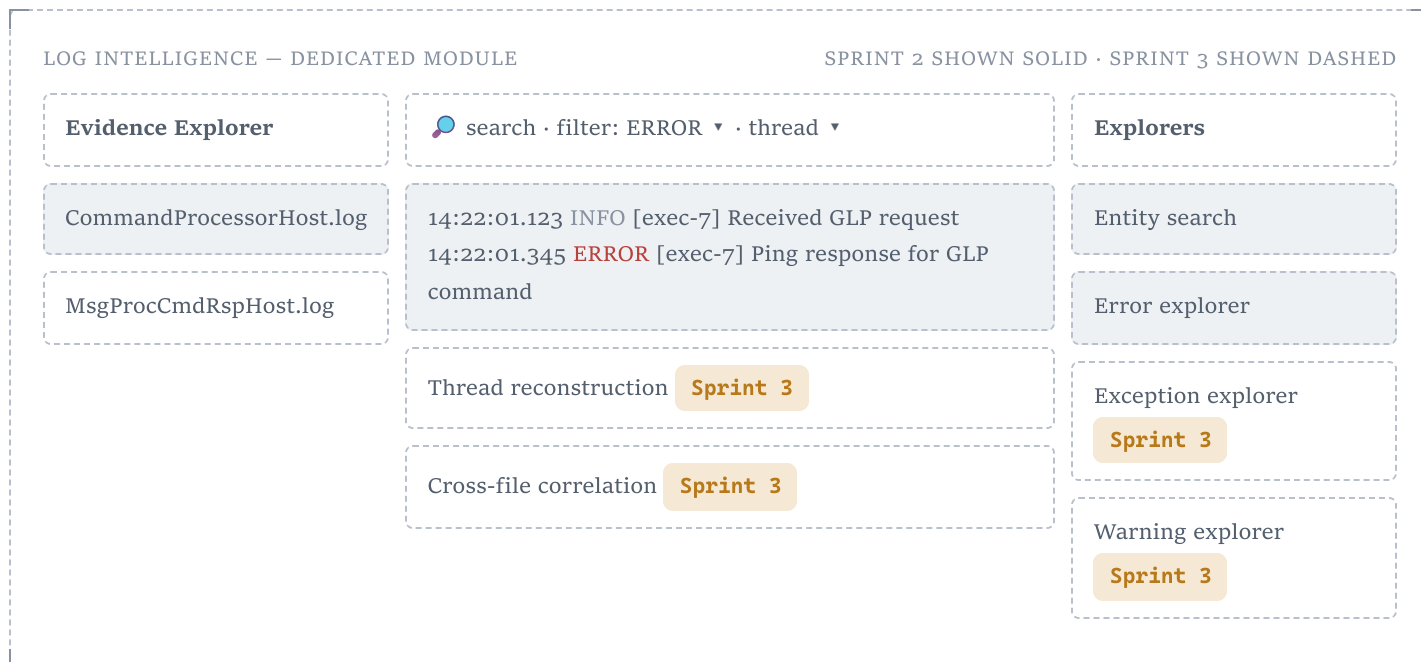
Cellular

PLC

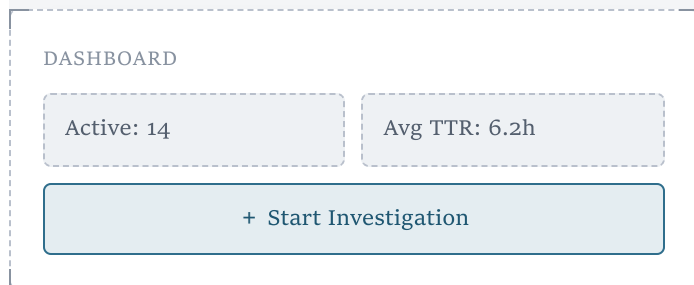
Blended

**Fig. 4b** — Pulls the matching Technology Registry profile (§03) into the workspace.

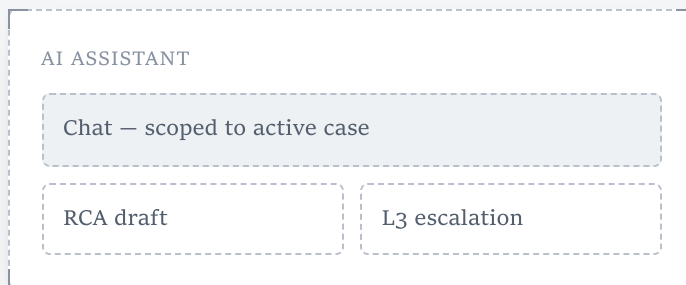
**Fig. 4a** — Category selection seeds the default playbook.



**Fig. 5** — All eight features you specified are designed here; four ship in Sprint 2, four build on top of the same viewer in Sprint 3 (see §11 roadmap note).



**Fig. 6a** — Hub, not the Assistant.



**Fig. 6b** — One module among eight, never the entry point (§01).

**Streamlit fit — unchanged assessment.** `st.navigation` + `st.tabs` cover the shell and tabs. Fig. 5's log viewer needs a custom HTML/JS component either way; Sprint 2 builds the foundation (search, filter, entity chips, error explorer) as that component, and Sprint 3 extends the same component rather than replacing it. A full Splunk-grade rewrite (virtualized scroll, multi-pane resize) stays a deliberate future decision, not a default.

## o8 Investigation experience

Unchanged mechanism from rev 1 — now explicitly reads Product Intelligence Engine data as one of its evidence signals.

### Investigation progress



"Historical Match" now requires both similarity *and* a non-contradicting Version Registry check — a 91% textual match with a version mismatch fills the step at reduced confidence, not full confidence.

### Investigation playbook

- ☒ ~~Verify firmware/DCW version~~
- ☒ ~~Check GEI sent after firmware upgrade~~
- ☐ Compare GLP payload against expected DCW
- ☐ Verify LP Set Number
- ☐ Review CommandProcessorHost

Steps are seeded from Wizard category *and* the matched Component Registry profile (§03) — "Review CommandProcessorHost" exists because the top match's `host_component` says so, not from a fixed template.

## 09 Revised folder structure

Additions from rev 1 are unchanged; new additions below are for Product Intelligence and recommendation reasoning.

```
app/
├─ domain/
│   └─ ... (unchanged: enums, entities, evidence, investigation)
│   └─ ingestion.py           # ParsedFile, FileKind enum
│   └─ playbook.py           # PlaybookStep, InvestigationProgress
│   └─ wizard.py             # IssueCategory, Technology enum
│   └─ recommendation.py     (modified) + MatchReasoningFactor
│   └─ product_intelligence.py # NEW - ComponentProfile, TechnologyProfile, VersionProfile
├─ engines/
│   └─ ingestion/            # Evidence Ingestion Pipeline
│   │   └─ file_type_registry.py
│   │   └─ parsers/ (text, docx, xlsx, pdf, image_ocr, evtx, zip)
│   └─ log_intelligence/
│   │   └─ ... (unchanged)
│   │   └─ ami_concepts.py    # + product_version entity type
│   └─ knowledge/
│   │   └─ ... (unchanged)
│   │   └─ reranker.py        # entity-overlap re-ranking
│   └─ investigation/
│   │   └─ engine.py          (unchanged)
│   │   └─ playbook_engine.py # seeds from wizard + component profile
│   └─ recommendation/
│   │   └─ engine.py          (modified) + builds MatchReasoning from Product Intelligence
│   │   └─ product_intelligence/ # NEW
│   │       └─ component_registry.py
│   │       └─ technology_registry.py
│   │       └─ version_registry.py
│   │       └─ engine.py      # facade - lookup + related_artifacts()
│   └─ sql_studio/           # stub - templates + suggested-query only
├─ infrastructure/
│   └─ db/ · vectorstore/    (unchanged)
│   └─ files/                # raw original blobs
├─ api/routers/
│   └─ investigations.py     (unchanged)
│   └─ ingestion.py · knowledge.py · sql_studio.py · wizard.py
│   └─ product_intelligence.py # NEW - component/technology/version lookup

data/sample_knowledge/
├─ historical_investigations.json (modified) + GLP/DCW mismatch record
├─ component_profiles.json        # NEW
├─ technology_profiles.json       # NEW
├─ version_profiles.json          # NEW

ui/
├─ Home.py + pages/*.py         (unchanged from rev 1 - 8 pages)
├─ components/
│   └─ evidence_explorer.py · progress_stepper.py · playbook_checklist.py
│   └─ log_viewer.py            # search/filter/entities in Sprint 2; correlation/threads in Sprint 3
│   └─ match_reasoning_panel.py  # NEW - renders the ✓/X factor list
```

## 10 Module responsibilities

| MODULE                                                                                                      | ANSWERS                             | BACKED BY                                                        | SPRINT 2 SCOPE                                                    |
|-------------------------------------------------------------------------------------------------------------|-------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------------|
|  Dashboard                 | What needs my attention?            | Investigation Engine                                             | Full                                                              |
|  Investigation Workspace   | What's the state of this case?      | Investigation + Playbook + Recommendation + Product Intelligence | Full                                                              |
|  Log Intelligence          | What actually happened in the logs? | Ingestion Pipeline + Log Intelligence Engine                     | Search/filter/entities/errors – correlation & threads in Sprint 3 |
|  Knowledge Center          | What do we already know?            | Knowledge Engine                                                 | Imported docs only                                                |
|  Historical Investigations | Has this happened before?           | Knowledge Engine + re-ranker                                     | Full search/filter                                                |
|  SQL Studio                | What query answers this?            | SQL Studio Engine (stub)                                         | Templates + suggested SQL only                                    |
|  AI Assistant              | Draft this for me                   | Recommendation Engine (templated, no LLM)                        | Scoped chat shell + templated drafts                              |
|  Settings                  | How is this configured?             | <code>app/config.py</code>                                       | Read-only                                                         |

Product Intelligence Engine has no nav item of its own — it's not one of your eight modules. It surfaces inside Investigation Workspace (component panel on the Overview tab) and Historical Investigations (component/technology/version filters).

11 Updated roadmap

Re-scoped from rev 1: Product Intelligence Engine and match transparency add real scope, so the deepest Log Intelligence features (thread reconstruction, cross-file correlation, exception/warning explorers) move to Sprint 3 rather than being compressed into Sprint 2 alongside a brand-new engine. Flagging this rather than quietly cramming it in — see §13.

| SPRINT | THEME                               | IN SCOPE                                                                                                                                                                                                  | EXPLICITLY EXCLUDED                                                           |
|--------|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| 2      | Redesign foundation                 | Nav shell, Ingestion Pipeline, Product/Technology/Version Intelligence, AMI entities, re-ranker + match reasoning, 2-step Wizard, Playbook Engine, basic Log Intelligence (search/filter/entities/errors) | Live connectors, thread reconstruction, cross-file correlation, SQL execution |
| 3      | Log Intelligence depth + SQL Studio | Thread reconstruction, cross-file correlation, exception/warning explorers, saved queries                                                                                                                 | Live ServiceNow, DB execution against prod                                    |
| 4      | Knowledge Center + AI Assistant     | Bulk PDF/PPT import, scoped chat, RCA/L2/L3 drafting                                                                                                                                                      | Live Wiki/ADO sync, external LLM calls (pending your review)                  |
| 5      | Connectors                          | ServiceNow first, then Wiki, then ADO                                                                                                                                                                     | Copilot integration                                                           |
| 6      | Production hardening                | Auth, PostgreSQL/SQL Server migration, multi-user                                                                                                                                                         | —                                                                             |

## 12 Sprint 2 plan

Rev 1's file list, plus Product Intelligence and match reasoning. No connector code anywhere in this list.

### PRODUCT INTELLIGENCE (NEW THIS REV)

**new** `app/domain/product_intelligence.py`

ComponentProfile, TechnologyProfile, VersionProfile

**new** `app/engines/product_intelligence/*.py`

Three registries + facade engine (lookup, related\_artifacts)

**new** `data/sample_knowledge/{component,technology,version}_profiles.json`

Seed data — CommandProcessorHost, RF Mesh family, CC 8.x/9.x at minimum

**new** `app/api/routers/product_intelligence.py`

Lookup endpoints for Workspace Overview + Historical Investigations filters

`app/domain/recommendation.py`

modified — add MatchReasoningFactor, extend Recommendation

`app/engines/recommendation/engine.py`

modified — build MatchReasoning by diffing against Product Intelligence Engine

### EVIDENCE INGESTION PIPELINE

**new** `app/domain/ingestion.py`

ParsedFile, FileKind, ParseWarning models

**new** `app/engines/ingestion/file_type_registry.py + parsers/*.py`

One parser per file family from §04

**new** `app/infrastructure/files/blob_store.py`

Raw original storage for preview/re-parse

## RECOMMENDATION RANKING

**new** `app/engines/log_intelligence/ami_concepts.py`

`command_type`, `host_component`, `firmware/dcw`, `GEI`, `LP set`, `product_version`

**new** `app/engines/knowledge/reranker.py`

Entity-overlap re-rank over top-N semantic candidates

`data/sample_knowledge/historical_investigations.json`

modified — add the GLP/firmware-DCW mismatch record

## INVESTIGATION EXPERIENCE

**new** `app/domain/playbook.py`, `app/domain/wizard.py`

`PlaybookStep`, `InvestigationProgress`, `IssueCategory`, `Technology`

**new** `app/engines/investigation/playbook_engine.py`

Seeds checklist from wizard + matched component profile

## UI — MULTI-PAGE SHELL

**new** `ui/Home.py` + `ui/pages/*.py`

`st.navigation` shell, 8 pages

**new** `ui/components/{evidence_explorer,progress_stepper,playbook_checklist,match_reasoning_panel}.py`

Shared widgets

**new** `ui/components/log_viewer.py`

Foundation only — search/filter/entity chips/error explorer (Sprint 3 extends it)

## TESTS

**new** `tests/test_file_type_registry.py`, `test_parsers.py`, `test_reranker.py`, `test_playbook_engine.py`, `test_product_intelligence.py`, `test_match_reasoning.py`

Same coverage discipline as Sprint 1

## 13 Sign-off

### New this revision – please confirm

- Product/Technology/Version Intelligence as **one engine, three registries**, not three separate engines – §03
- Log Intelligence split across two sprints: search/filter/entities/errors in Sprint 2, thread reconstruction + cross-file correlation + exception/warning explorers in Sprint 3 – §07 Fig. 5, §11
- Match reasoning is a **rule-based structured diff** against Product Intelligence data, not an LLM-generated explanation – §05

### Carried over from rev 1 – still open

- OCR as best-effort, not a hard dependency – §04
- Custom Streamlit component for the log viewer, vs. a later frontend rewrite – §07
- No LLM/external API calls in AI Assistant for Sprint 2 – §11, Sprint 4 note